

CSC633 MACHINE LEARNING



TEXAS A&M UNIVERSITY

Department of Computer
Science & Engineering

CSCE 633: Machine Learning

Reinforcement Learning

Texas A&M University

Topics

- What is reinforcement learning?
- An example application and Markov Decision Process
- Value Iteration
- Q Learning (and Policy Iteration)

Q-learning

- $Q^*(s, a)$ - the expected sum of rewards from being at s , taking action a and then following π^*
- $\pi^*(s) \leftarrow \underset{a}{\operatorname{argmax}}[Q^*(s, a)]$
- Learning Q values with dynamic programming?
 - Similar to value iteration



Courtesy: Dr. Sharon

Q-learning by value iteration

- $V^*(s) = \max_a [\sum_{s'} P(s'|s, a) (R(s, a, s') + \gamma V^*(s'))]$
- $V^*(s) = \max_a [Q^*(s, a)]$
- $Q^*(s, a) = \sum_{s'} P(s'|s, a) (R(s, a, s') + \gamma V^*(s'))$
- $Q^*(s, a) = \sum_{s'} P(s'|s, a) (R(s, a, s') + \gamma \max_a [Q^*(s', a)])$
- Solve iteratively
 - $Q_{k+1}(s, a) = \sum_{s'} P(s'|s, a) (R(s, a, s') + \gamma \max_a [Q_k(s', a)])$
 - Can use Asynchronous DP

Courtesy: Dr. Sharon

Issue 3: The policy often converges long before the values

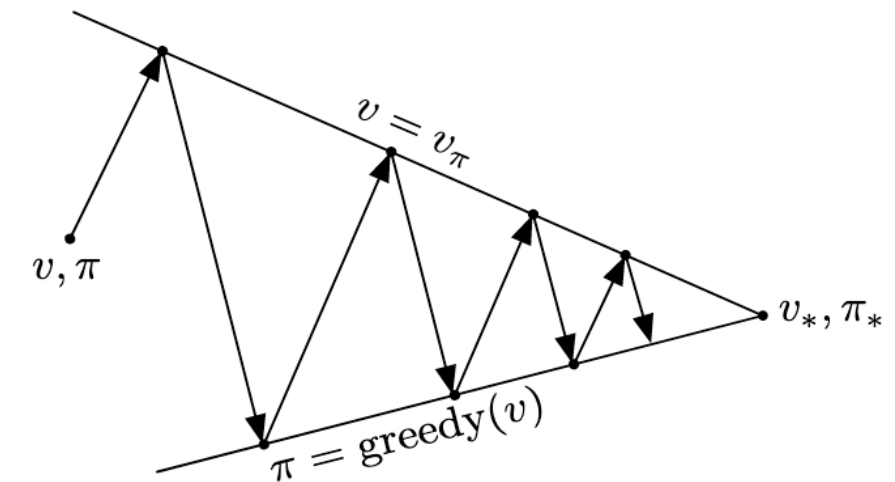
- Value iteration converges to the true utility value: $V_{k \rightarrow \infty} \rightarrow V^*$
- V^* implies the optimal policy: π^*
- Can we converge directly on π ?
 - Improve the policy in iteration until reaching the optimal one



Courtesy: Dr. Sharon

Policy Iteration

1. **Compute V_π** : calculate state value for some fixed policy (not optimal values)
2. **Update π** : update policy using one-step look-ahead with resulting (non optimal) values
3. Repeat steps until policy converges (optimal values)



- Converges to π^*
 - $\forall s, V_{k \geq 0}(s) \leq V_{k+1}(s)$ i.e., π_i improves monotonically with i
 - A fixed point, $\forall s, V_k(s) = V_{k+1}(s)$, implies π^*

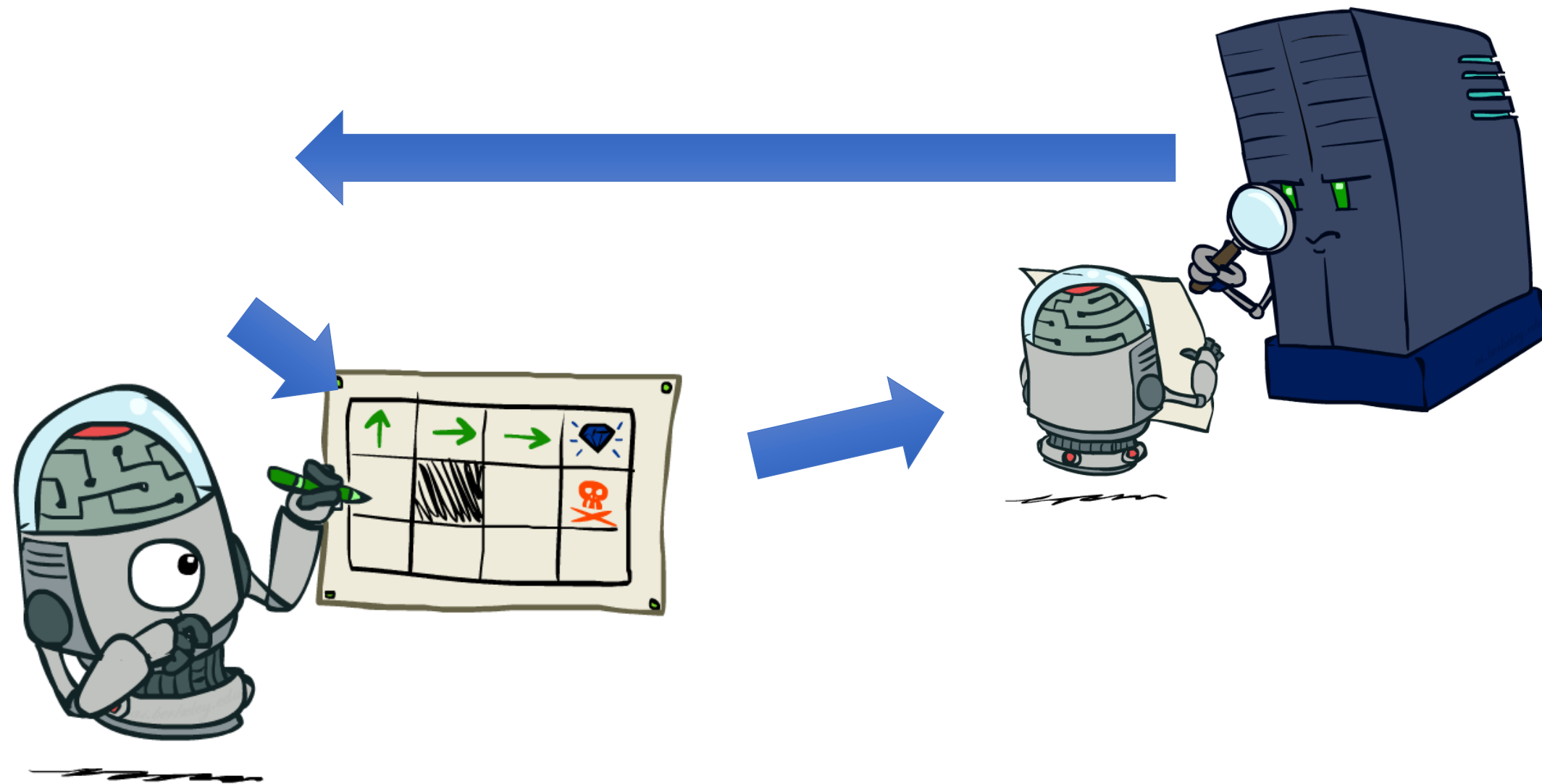
Courtesy: Dr. Sharon

Policy Evaluation

- Why is calculating V_π easier than calculating V^* ?
 - Turns non-linear Bellman equations into linear equations
- $v^*(s) = \max_a [\sum_{s'} P(s'|s, a) (R(s, a, s') + \gamma v^*(s'))]$
- $v_\pi(s) = \sum_{s'} P(s'|s, \pi(s)) (R(s, \pi(s), s') + \gamma v^*(s'))$
- Solve a set of linear equations in $O(S^2)$
 - Solve with Numpy (numpy.linalg.solve)
 - See: <https://numpy.org/doc/stable/reference/generated/numpy.linalg.solve.html#numpy.linalg.solve>

Courtesy: Dr. Sharon

Policy iteration



Courtesy: Dr. Sharon

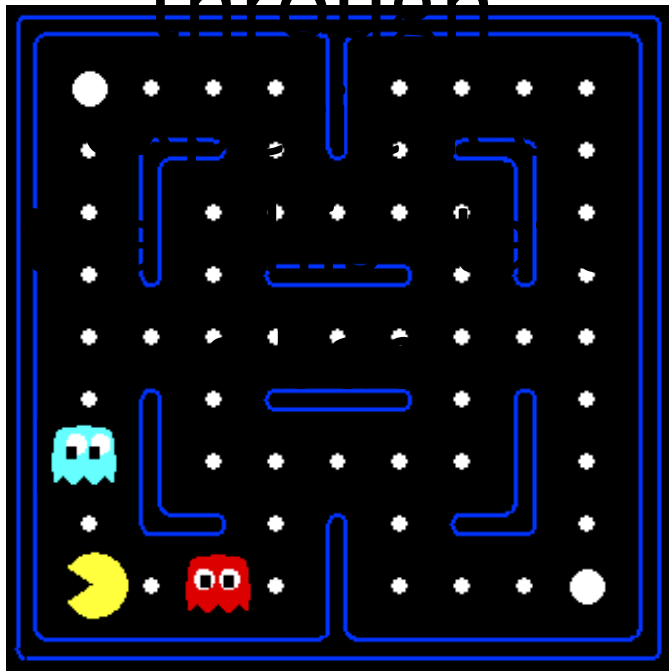
Comparison

- Both value iteration and policy iteration compute the same thing (all optimal values)
- In value iteration:
 - Every iteration updates both the values and (implicitly) the policy
 - We don't track the policy, but taking the max over actions implicitly recomputes it
- In policy iteration:
 - We do several passes that update utilities with fixed policy (each pass is fast because we consider only one action, not all of them)
 - After the policy is evaluated, a new policy is chosen (slow like a value iteration pass)
 - The new policy will be better (or we're done)

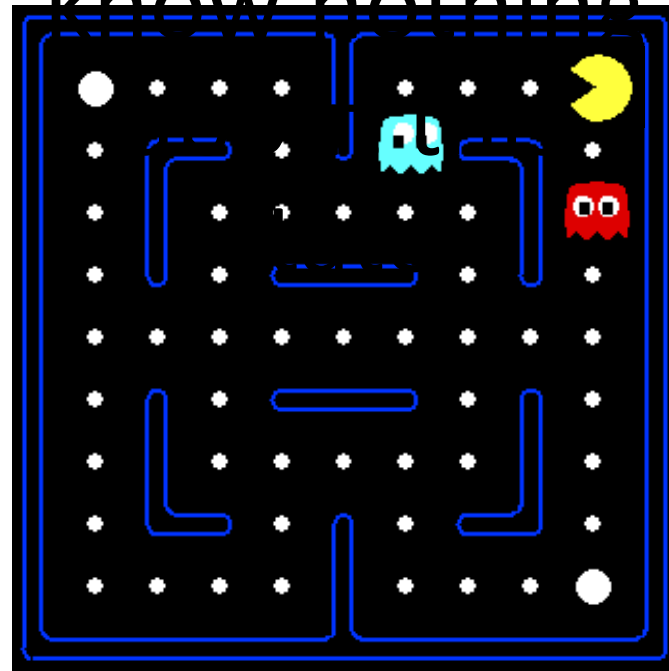
Courtesy: Dr. Sharon

Example: Pacman

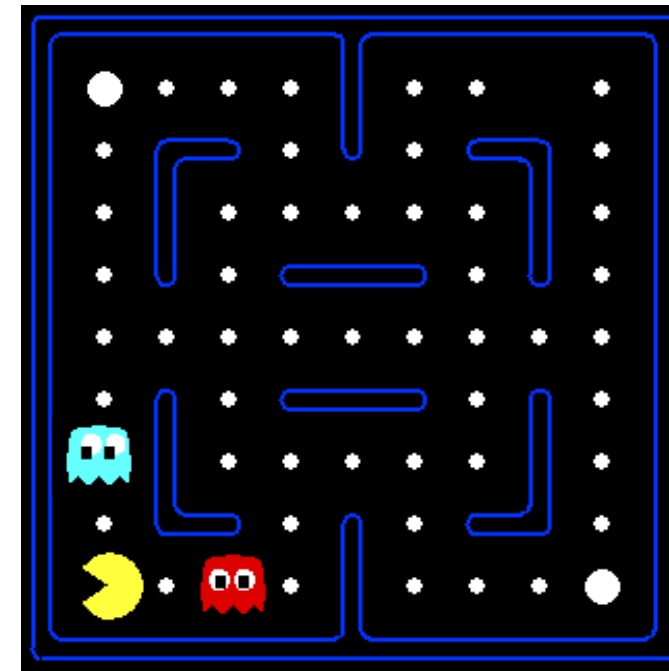
Let's say we
discover
through



In naïve q-
learning, we
know nothing



Or even this
one!



We must generalize our knowledge!

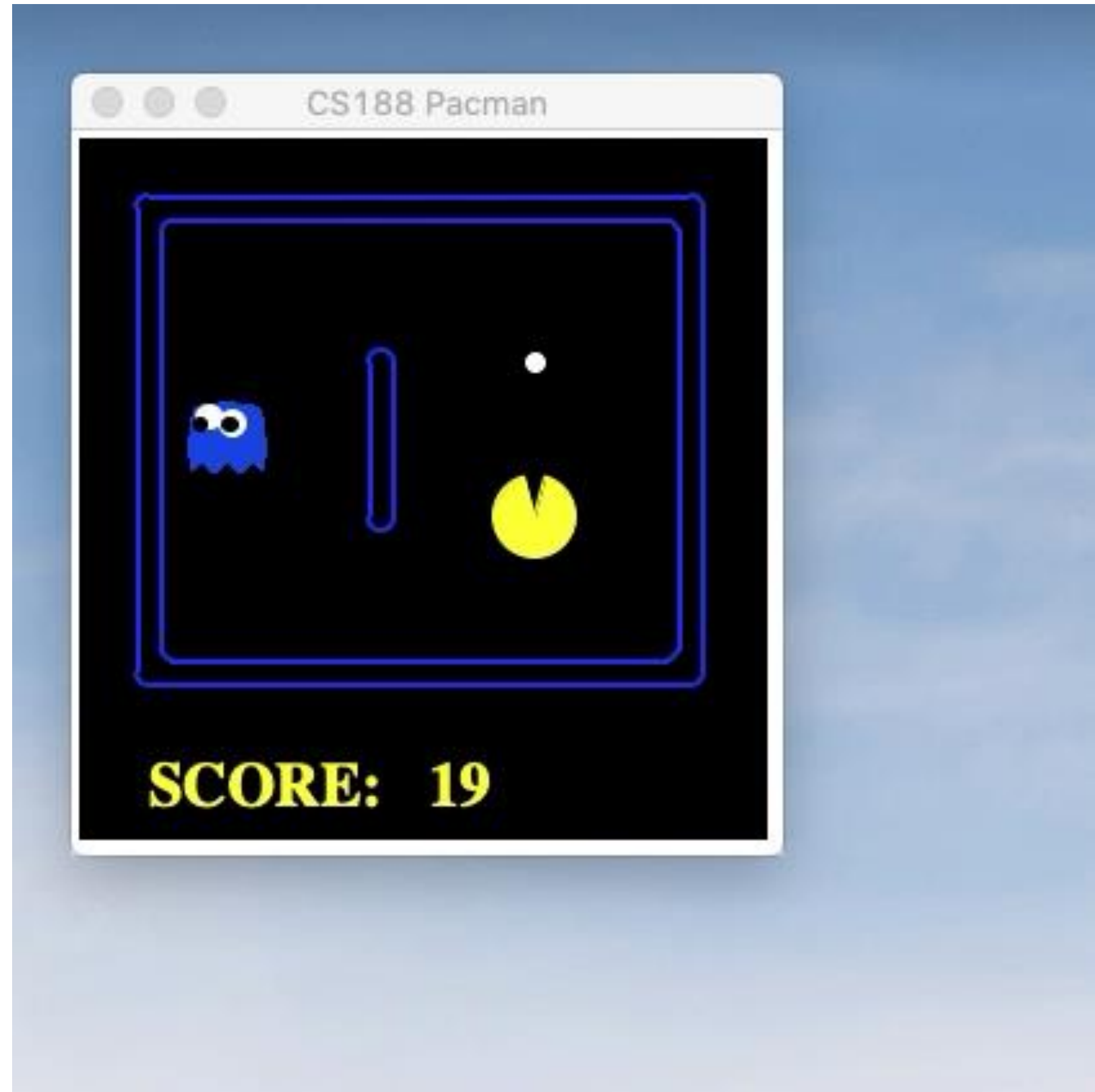
Courtesy: Dr. Sharon

- Naïve Q-learning
- After 50 training episodes



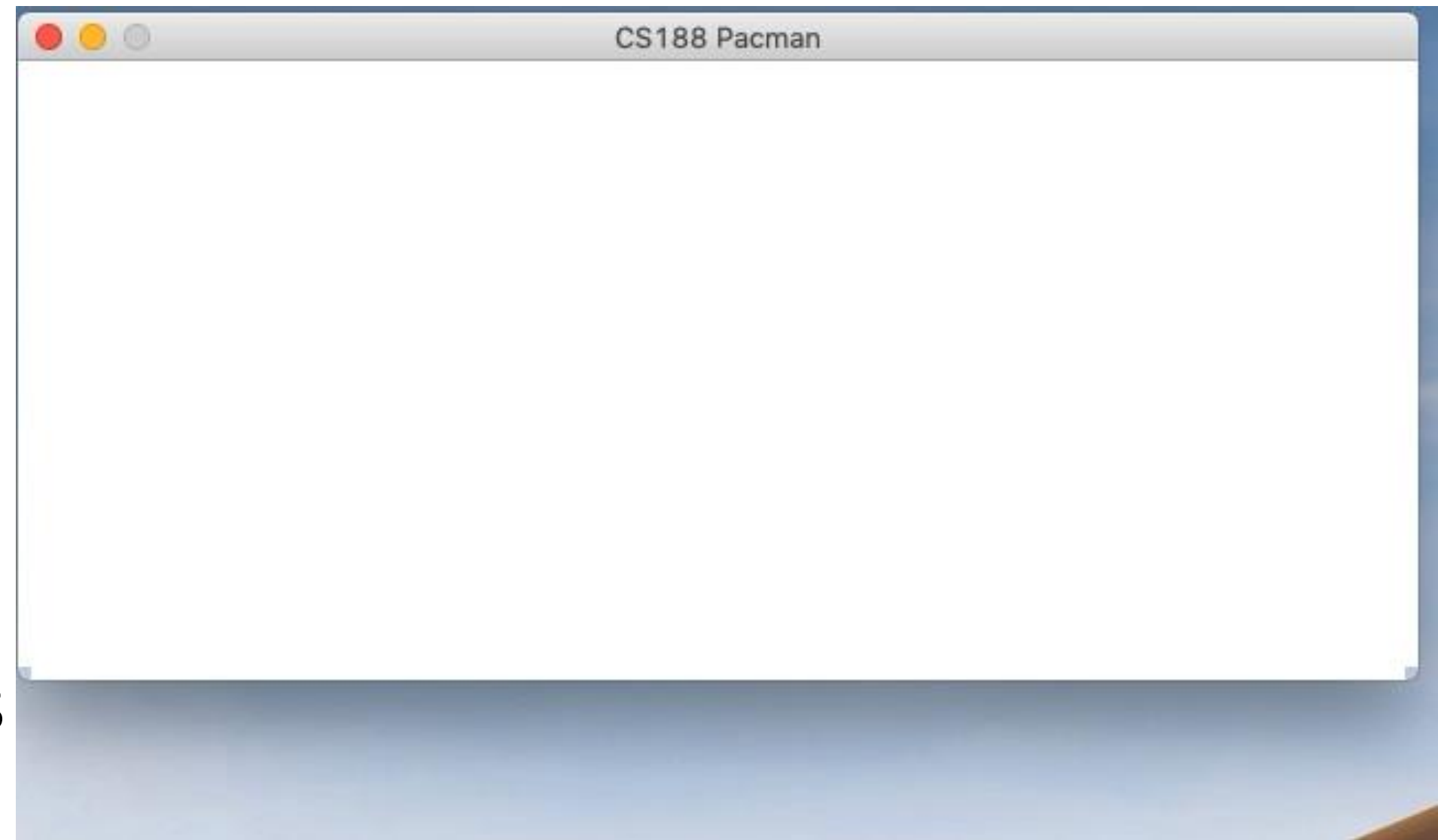
Courtesy: Dr. Sharon

- Generalize Q-learning with function approximator



Courtesy: Dr. Sharon

- Generalizing knowledge results in efficient learning
- E.g., learn to avoid the ghosts



Courtesy: Dr. Sharon

Generalizing with Deep learning

- **Supervised:** Require large amounts of hand-labelled training data
 - **RL** on the other hand, learns from a scalar reward signal that is frequently sparse, noisy and delayed
- **Supervised:** Assume the data samples are independent
 - In **RL** one typically encounters sequences of highly correlated state
- **Supervised:** Assume a fixed underlying distribution
 - In **RL** the data distribution changes as the algorithm learns new behaviors
- DQN was first to demonstrate that a convolutional neural network can overcome these challenges to learn successful control policies from raw video data in complex RL environments

Courtesy: Dr. Sharon

Deep Q learning [Mnih et al. 2015]

- Create a generic neural network based agent that is able to successfully learn to operate in as many domains as possible
- The network is not provided with any domain-specific information or hand-designed features
- Must learn from nothing but the raw input (pixels), the reward, terminal signals, and the set of possible actions

Courtesy: Dr. Sharon

Original Q-learning

- What's the problem with Basic Q-learning?
 1. Train on correlated successive states
 2. Train on sparse data, one sample per state
 3. Policy change -> sample distribution mismatch

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

```
Repeat (for each episode):  
  Initialize  $S$   
  Repeat (for each step of episode):  
    Choose  $A$  from  $S$  using policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)  
    Take action  $A$ , observe  $R, S'$   
     $\theta = \theta + \alpha \left( R + \gamma \max_a \hat{Q}(S', a; \theta) - \hat{Q}(S, A; \theta) \right) \nabla_{\theta} \hat{Q}(S, A; \theta)$   
     $S \leftarrow S'$   
  until  $S$  is terminal
```

Courtesy: Dr. Sharon

Deep Q learning [Mnih et al. 2015] (1)

- DQN addresses problems of correlated data and non-stationary distributions
 - Use an experience replay mechanism
 - Randomly samples and trains on previous transitions
 - Results in a smoother training distribution over many past behaviors

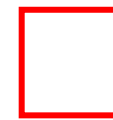
Courtesy: Dr. Sharon

Deep Q learning [Mnih et al. 2015] (2)

- Learn a function approximator (Q network), $\hat{Q}(s, a; \theta) \approx Q^*(s, a)$
- Value propagation: $Q^*(s, a) = \mathbb{E}_{s' \sim \mathcal{E}}[r + \gamma \max_{a'} Q^*(s', a')]$
 - $\mathcal{E}$ represents the environment (underlying MDP)
- Update $\hat{Q}(s, a; \theta)$ at each step i using SGD with squared loss:
- $L_i(\theta_i) = \mathbb{E}_{s, a \sim \rho(\cdot)} \left[\left(y_i - \hat{Q}(s, a; \theta_i) \right)^2 \right]$
 - $y_i = \mathbb{E}_{s' \sim \mathcal{E}} \left[r + \gamma \max_{a'} \hat{Q}(s', a'; \theta_{i-1}) \right]$
 - $\rho(s, a)$ is the behavior distribution, e.g., epsilon greedy
 - θ_{i-1} is considered fix when optimizing the loss function (helps when taking the derivative with respect to θ)

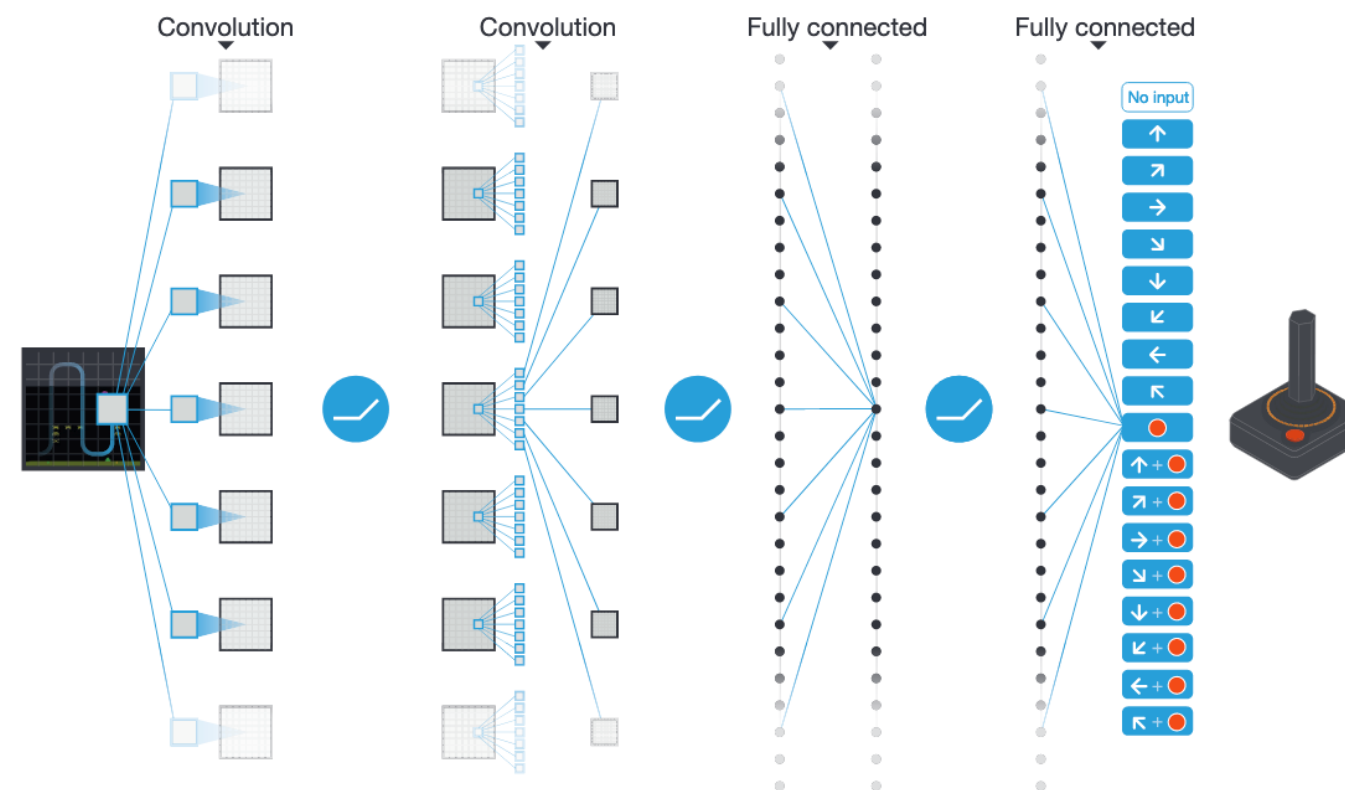
Courtesy: Dr. Sharon

Deep Q learning [Mnih et al. 2015] (3)



Independent of θ_i (because θ_{i-1} is considered fix)

- $L_i(\theta_i) = \mathbb{E}_{s,a \sim \rho(\cdot)} \left[\left(y_i - \hat{Q}(s, a; \theta_i) \right)^2 \right]$
- $\nabla_{\theta_i} L_i(\theta_i) = \mathbb{E}_{s,a \sim \rho(\cdot)} \left[\left(r + \gamma \max_{a'} \hat{Q}(s', a'; \theta_{i-1}) - \hat{Q}(s, a; \theta_i) \right) \nabla_{\theta_i} \hat{Q}(s, a; \theta_i) \right]$



Courtesy: Dr. Sharon

Conclusion: RL itself is a complex and fascinating field

